

# ¿Cuándo Diseñamos? + Evaluación

*BDUF/YAGNI/JEDUF · emergent vs intentional · guardrails · platform eng · SAAM/ATAM*

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

## RESUMEN

Dos preguntas de proceso arquitectónico. **¿Cuándo diseñamos?** — la cátedra defiende **JEDUF**: diseñar lo irreversible, dejar emerger lo barato, encauzar lo no decidido con *guardrails* y golden paths. **¿Cómo evaluamos lo diseñado?** — métodos basados en escenarios: SAAM (1993) → ATAM (gold standard) → Lightweight ATAM (pragmático).

## B1 ¿Cuándo diseñamos?

Eje en disputa: "¿cuánto diseñamos antes de escribir código?" es un espectro, no un binario. Tres posturas canónicas lo balizan (source: bduf-yagni-jeduf.md).

**BDUF · YAGNI · JEDUF**

### DEFINICIÓN

**BDUF (Big Design Up Front)**: diseñar todo antes de codear. Arrastre del waterfall. — **YAGNI (You Ain't Gonna Need It)**: no diseñar para necesidades futuras; viene de XP/Agile (Beck, Jeffries). — **JEDUF (Just Enough Design Up Front)**: diseñar lo **suficiente** para arrancar con dirección y dejar emerger el resto (source: bduf-yagni-jeduf.md).

### CUÁNDO NO · CUIDADO

**BDUF** — **cuándo NO**: asume dominio entendido al inicio (contradice el cono de incertidumbre); diseño "de papel" que no sobrevive la realidad. **Paradoja**: más diseño cuando menos se sabe. Anti-patrones: Analysis Paralysis, Architecture Astronauts (Spolsky).

### CUÁNDO NO · CUIDADO

**YAGNI** — **cuándo NO**: llega **tarde** a decisiones irreversibles (DB schema, API pública, modelo de seguridad, multi-tenancy). Anti-patrón: Big Ball of Mud (Foote, Yoder). No es "no pensar" — es "no implementar sin necesidad".

### CUÁNDO SÍ · VENTAJAS

**JEDUF** — **postura de la cátedra**: anclar decisiones de alto costo de cambio (persistencia, contratos externos, seguridad, multi-tenancy); aplazar las de bajo costo (nombres internos, packages, libs utilitarias) (source: bduf-yagni-jeduf.md).

#### REGLA DE ORO

**Heurística de reversibilidad (Bezoz):** si la decisión es *one-way door* → diseñar. Si es *two-way door* → iterar. Calibrar "just enough" = costo de revertir × probabilidad de tener que revertir.

TABLA 1. BDUF vs YAGNI vs JEDUF

| Dimensión        | BDUF                                | YAGNI                                    | JEDUF                              |
|------------------|-------------------------------------|------------------------------------------|------------------------------------|
| Diseño inicial   | Exhaustivo                          | Mínimo o nulo                            | Sólo lo irreversible               |
| Asume dominio    | Alto                                | Bajo                                     | Medio                              |
| Riesgo principal | Diseño obsoleto al entregar         | Deuda arquitectónica tardía              | Definir mal qué es "suficiente"    |
| Encaja en        | Dominios regulados, contratos fijos | Startups exploratorias, greenfield chico | Mayoría de proyectos empresariales |
| Evaluación       | ATAM completo                       | Ninguna formal                           | Lightweight ATAM                   |

*JEDUF patológico: el criterio de "suficiente" queda en una sola persona sin consenso → BDUF o YAGNI encubierto según el temperamento del arquitecto (source: bduf-yagni-jeduf.md).*

## Emergent vs Intentional design

JEDUF dice *cuánto* diseñar; emergent/intentional son los dos **mecanismos** que producen la arquitectura (source: emergent-vs-intentional-design.md).

### Emergent design

La arquitectura **surge** del refactoring continuo. Requiere: tests exhaustivos, disciplina de refactor, equipo chico y colocado, dominio simple. No escala con el tamaño del equipo ni anticipa lo irreversible; degenera en Big Ball of Mud si falla la disciplina.

### Intentional design

La arquitectura se **define deliberadamente** con artefactos (ADRs, diagramas, árbol de utilidad). Reduce riesgo en lo caro de revertir y da lenguaje común; puede divergir del código real y tender a BDUF.

#### FUENTE

*Fowler (Is Design Dead?, 2004): emergent design **no es "no diseñar"** — es diseñar con otro mecanismo. Requiere test coverage alta, refactoring continuo y principios internalizados (SOLID, smells). Es decir: **emergent design requiere disciplina intentional** (source: emergent-vs-intentional-design.md).*

#### EJEMPLO

**Mapeo:** BDUF = intentional puro a priori · YAGNI = emergent puro · **JEDUF = intentional para lo caro + emergent para lo barato**. JEDUF es el nombre práctico de la combinación madura (source: emergent-vs-intentional-design.md).

## Architectural guardrails

### DEFINICIÓN

**Guardrails:** reglas organizacionales que **encauzan** decisiones arquitectónicas locales **sin dictar** la solución. Punto medio entre arquitectura prescriptiva (top-down) y laissez-faire. Metáfora literal: barreras de contención en la autopista — no dictan por dónde manejas, evitan que te salgas de la ruta (source: architectural-guardrails.md).

JEDUF no dice *qué* decidir, sólo *cuánto*. Los guardrails encauzan lo no-diseñado-arriba sin caer en YAGNI puro (source: bduf-yagni-jeduf.md). Ejemplos: `/metrics` Prometheus obligatorio, `trace-id` OpenTelemetry, OAuth2 con lib de plataforma (prohibido JWT a mano), PII encriptada at-rest, SLOs antes del launch.

### CUÁNDO SÍ • VENTAJAS

**Guardrails buenos:** pocos ( $20 > 200$ ); **justificados** (cada uno con su "por qué"); **automatizables** (lint/CI, no disciplina manual); **revisables** (proceso de excepción).

### CUÁNDO NO • CUIDADO

**Guardrails malos / anti-patronos:** prescribir tech stack ("todos React") = arquitectura prescriptiva, no guardrail; inflación (1 incidente → 1 regla → 300 reglas); sin plataforma que verifique; burocráticos (6 aprobaciones por excepción).

*Distinción operativa: **Guardrail** (decisión arq. a nivel org, mandatorio con excepción documentada) ≠ **Standard** (cómo implementar, recomendación fuerte) ≠ **Policy** (compliance/legal, mandatorio sin excepción) (source: architectural-guardrails.md).*

## Platform engineering + golden paths

### DEFINICIÓN

**Platform Engineering:** disciplina que construye y opera **Internal Developer Platforms (IDPs)** — productos internos que reducen la **carga cognitiva** del dev de aplicación, materializando los guardrails como herramientas consumibles self-service (source: platform-engineering.md). **No es DevOps renombrado:** DevOps es la filosofía, Platform Engineering el equipo/producto que la implementa a escala.

### DEFINICIÓN

**Golden Path (paved road):** camino feliz pre-construido para una tarea común (crear servicio, agregar DB, exponer API). Opción **default + fácil + bendecida + observada**. Popularizado por Spotify (~2018) (source: golden-paths.md).

Relación clave: el guardrail es la **regla** ("todo servicio expone `/metrics`"); el golden path es la **encarnación operacional** que la cumple sin pensar (al crear el servicio ya viene con métricas). La IDP convierte guardrails de *doc* a *ley física* del ecosistema interno (source: golden-paths.md).

#### EJEMPLO

**Ejemplos vistos en clase:** **Fury** (MercadoLibre), **Backstage** (Spotify → CNCF), **Jarvis**. Conceptos hermanos: *single pane of glass* (una UI con deploys, logs, métricas, costos, alertas) y *self-service* ("you build it, you run it, we pave the road") (source: platform-engineering.md).

#### REGLA DE ORO

**Regla del platform engineering sano:** el golden path es *golden*, no *forzoso* — debe permitir **opt-out justificado**. Si desviarse es más fácil que seguirlo, la plataforma fracasó (source: golden-paths.md).

#### CUÁNDO NO • CUIDADO

**Cuándo NO platform engineering:** en orgs chicas (<20 devs) es **overkill** — un par de convenciones y scripts alcanzan. Anti-patrones: *ivory tower* (desconectado de devs reales), *plataforma como prisión* (cero flexibilidad → shadow IT), *free-for-all* (ofrece todo, nadie sabe qué usar) (source: platform-engineering.md).

## B2 Evaluación de arquitecturas

---

Evaluar ≠ diseñar. Estos métodos **no producen** arquitectura: la analizan contra sus atributos de calidad usando escenarios. Linaje: SAAM (1993) → ATAM (2000) → Lightweight ATAM (source: saam.md, atam.md).

### SAAM (1993)

#### DEFINICIÓN

**SAAM (Software Architecture Analysis Method):** primer método formal de evaluación basado en escenarios. Kazman, Bass, Abowd, Webb — ICSE 1993. Ancestro directo de ATAM. Responde: *"¿esta arquitectura satisface los escenarios que importan, y qué tan costoso sería extenderla?"* (source: saam.md).

**Pasos:** (1) caracterizar forma canónica de la arquitectura; (2) desarrollar escenarios; (3) clasificarlos en directos (soportados as-is) vs indirectos (requieren cambios); (4) evaluar cada uno (costo de cambio en los indirectos); (5) detectar interacciones → *hot spots*; (6) sintetizar riesgos y trade-offs (source: saam.md).

#### EJEMPLO

**Escenarios SAAM:** de **uso** ("el usuario hace X, el sistema responde Y") y de **cambio** ("si mañana hay que soportar Z, ¿qué se modifica?"). Los de cambio son los más informativos: miden mantenibilidad y evolución. Output: matriz escenario × componente + componentes "hot" (source: saam.md).

#### CUÁNDO NO • CUIDADO

**Crítica a SAAM:** sesgo hacia **modificabilidad** (performance/security menos sistemáticos); no captura trade-offs entre atributos explícitamente como ATAM; garbage in, garbage out con los escenarios (source: saam.md).

### ATAM — gold standard

#### DEFINICIÓN

**ATAM (Architecture Tradeoff Analysis Method):** método formal para evaluar una arquitectura frente a sus atributos de calidad. Kazman, Klein, Clements — SEI, Carnegie Mellon, 2000. **Gold standard** del área (source: atam.md).

**Outputs:** árbol de utilidad con escenarios priorizados; *sensitivity points*; *tradeoff points*; risks / non-risks; risk themes (patrones transversales que apuntan a problemas sistémicos).

TABLA 2. ATAM — 4 fases, 9 pasos

| Fase                                     | Pasos                                                                                                                                                                                                | Quiénes                        |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>0 — Partnership &amp; Preparation</b> | Alinear stakeholders, formar equipo evaluador (3-5 externos), confirmar scope/logística                                                                                                              | Líderes + evaluadores          |
| <b>1 — Evaluation (Round 1)</b>          | 1 presentar ATAM · 2 business drivers · 3 presentar arquitectura · 4 identificar approaches · 5 generar árbol de utilidad (peso: importancia × dificultad) · 6 analizar approaches vs escenarios top | Sólo arquitectos + evaluadores |
| <b>2 — Evaluation (Round 2)</b>          | 7 brainstorming y priorización de escenarios · 8 re-analizar approaches · 9 presentar resultados (risks, sensitivity, tradeoffs, themes)                                                             | Stakeholders ampliados (10-20) |
| <b>3 — Follow-up</b>                     | Reporte formal, plan de mitigación de riesgos, lecciones aprendidas                                                                                                                                  | Equipo evaluador               |

#### TRADE-OFF

**Sensitivity vs Tradeoff (clave del método):** **Sensitivity point** = decisión de la que depende fuertemente *un* atributo (ej: broker central sensible a **PERFORMANCE** — si satura, el atributo cae). **Tradeoff point** = decisión que toca *varios* atributos en direcciones opuestas (ej: replicar el broker mejora **AVAILABILITY** pero empeora consistency → eventual consistency) (source: atam.md).

#### CUÁNDO NO · CUIDADO

**Costo / cuándo NO:** **20-30 person-days** en ~4-6 semanas; equipo evaluador externo + 10-20 stakeholders. Inviabile para la mayoría de equipos. Es una **foto**, no monitoreo continuo. Se reserva para decisiones de **alto impacto e irreversibles** (source: atam.md).

#### EJEMPLO

**Kata de cátedra:** Healthcare.gov (Clase V) — con el reporte OEI post-mortem, reconstruir el árbol de utilidad que CMS *debería* haber armado en 2012-2013: qué atributos se sub-priorizaron, qué sensitivity points se ignoraron, qué tradeoffs quedaron implícitos (source: atam.md).

## Lightweight ATAM

#### DEFINICIÓN

**Lightweight ATAM:** versión **comprimida y pragmática** del ATAM formal para equipos chicos o iteraciones ágiles donde no se justifican 20-30 person-days. Mantiene el **espíritu** (árbol de utilidad, sensitivity, tradeoffs) reduciendo la carga de proceso: **1-2 días de workshop**, equipo + 1-2 stakeholders clave, reporte interno (source: lightweight-atam.md).

#### CUÁNDO SÍ • VENTAJAS

**Cuándo SÍ:** iteración agile (revalidar cada 3-6 meses); equipos <10 personas; checkpoint temprano antes de que cambiar sea caro; warm-up antes de un ATAM completo.

#### CUÁNDO NO • CUIDADO

**Cuándo NO:** decisiones de alto impacto e irreversibles (sistema crítico regulado → ATAM completo, cf. Healthcare.gov); stakeholders diversos con conflictos (necesitan el Round 2 formal para dirimir).

#### CUÁNDO NO • CUIDADO

**Riesgos de la versión liviana:** el equipo se valida a sí mismo (falta diversidad de stakeholders); escenarios sesgados sin challenge externo; documentación flaca. **Mitigar:** invitar ≥1 stakeholder externo (otro arquitecto, SRE, seguridad); usar plantilla estándar de reporte; agendar el próximo antes de cerrar éste (source: lightweight-atam.md).

### Tabla comparativa

TABLA 3. SAAM vs ATAM vs Lightweight ATAM

| Dimensión               | SAAM (1993)                        | ATAM (2000)                                         | Lightweight ATAM                     |
|-------------------------|------------------------------------|-----------------------------------------------------|--------------------------------------|
| <b>Foco</b>             | Modificabilidad / usabilidad       | Todos los atributos de calidad                      | Decisiones más riesgosas             |
| <b>Estructura</b>       | Single-round, liviano              | 4 fases, 9 pasos                                    | Workshop comprimido                  |
| <b>Costo / duración</b> | Días                               | 20-30 person-days (4-6 sem.)                        | 1-2 días                             |
| <b>Stakeholders</b>     | Equipo técnico                     | Amplio (negocio + técnico, 10-20)                   | Equipo + 1-2 clave                   |
| <b>Output</b>           | Matriz escenarios × componentes    | Utility tree, sensitivity, tradeoffs, risks, themes | Reporte interno del equipo           |
| <b>Estado / uso</b>     | Precursor histórico; evals rápidas | Gold standard; alto impacto e irreversible          | Iteración agile, checkpoint temprano |

#### REGLA DE ORO

**Regla de selección:** arquitectura joven o comparar A vs B → **SAAM**. Iteración incremental, equipo chico → **Lightweight**. Sistema crítico, irreversible, regulado → **ATAM completo** (source: atam.md, saam.md, lightweight-atam.md).

#### FUENTE

Ford, Parsons, Kua (*Building Evolutionary Architectures*): reemplazar evaluaciones puntuales por **fitness functions** — chequeos automáticos y continuos en CI. Complementan a *Lightweight ATAM*: las *fitness functions* verifican lo conocido, el workshop identifica lo nuevo (source: lightweight-atam.md).

#### EN EL PARCIAL

El proceso encadena: JEDUF decide *qué* diseñar (lo irreversible) → guardrails + golden paths encauzan lo demás → ATAM/Lightweight/SAAM *evalúan* lo diseñado. Errores que matan la respuesta: confundir ATAM con un método de diseño (sólo evalúa); olvidar la distinción **sensitivity (1 atributo) vs tradeoff (varios, en conflicto)**; recomendar ATAM completo para un equipo chico (no justifica 20-30 person-days).